

Практическое занятие №2

Электронный журнал оценок студентов

Разработанное приложение представляет собой модульный фреймворк на Node.js, который позволяет загружать и выполнять модули расширения с автоматическим управлением их зависимостями. Предметная область — электронный журнал оценок студентов: модули принимают записи об успеваемости, проверяют их корректность, сохраняют в общем хранилище, экспортируют в файл и формируют сводный отчёт.

При запуске приложение читает конфигурационный файл `config/modules.json`, загружает указанные в нём модули из папки `modules`, проверяет корректность их зависимостей, определяет порядок запуска и последовательно выполняет все зарегистрированные модулями действия. В результате работы создаётся файл `grades.txt` с данными об оценках и выводится информация о выполненных операциях.

Фреймворк построен на принципе инверсии управления: ядро управляет жизненным циклом модулей, а модули только реализуют заданный контракт. Это позволяет добавлять новые модули без изменения кода ядра.

Состав ядра

Ядро состоит из следующих компонентов:

- `container.js` — контейнер внедрения зависимостей. Поддерживает два типа времени жизни объектов: Singleton (один экземпляр на всё приложение) и Transient (новый экземпляр при каждом запросе). Позволяет регистрировать сервисы и получать их по ключу.
- `moduleLoader.js` — загрузчик модулей. Выполняет динамическую загрузку модулей из папки `modules`, проверяет наличие всех зависимостей, строит порядок запуска с помощью топологической сортировки, обнаруживает циклические зависимости и проверяет совместимость версий контракта.
- `main.js` — точка входа. Орkestрирует весь процесс: загружает модули, определяет порядок запуска, вызывает регистрацию сервисов, инициализацию модулей и выполняет зарегистрированные действия.
- `errors.js` — определяет класс `ModuleLoadError` для ошибок, связанных с загрузкой модулей.
- `logger.js` — утилита для цветного логирования в консоль.

Состав модулей

Все модули следуют единому контракту: каждый экспортирует объект с полями `name`, `version`, `contractVersion`, `requires` и методами `register()` и `init()`. Реализовано 7 модулей:

- `core.js` — базовый модуль. Регистрирует сервисы `clock` (время) и `storage` (хранилище записей об оценках). Не имеет зависимостей.
- `logging.js` — модуль журналирования. Регистрирует действие, которое фиксирует факт открытия журнала с отметкой времени. Зависит от Core.

- **validation.js** — модуль проверки оценок. Регистрирует действие, которое валидирует тестовый набор записей (имя студента, предмет, оценка от 2 до 5) и добавляет их в хранилище. Зависит от Core.
- **export.js** — модуль экспорта. Регистрирует действие, которое сохраняет данные из хранилища в файл **grades.txt**. Зависит от Core и Validation.
- **report.js** — модуль сводки. Регистрирует действие, которое выводит количество записей и средний балл. Зависит от Core и Export.
- **di-demo.js** — демонстрационный модуль. Показывает, как зависимости внедряются через конструктор с помощью контейнера DI. Зависит от Core.
- **lifetime-demo.js** — демонстрационный модуль. Показывает разницу между Singleton и Transient. Зависит от Core.

Демонстрация работы приложения

Запуск приложения командой **npm start** приводит к загрузке всех указанных в конфигурации модулей, построению корректного порядка запуска с учётом зависимостей и последовательному выполнению зарегистрированных действий. На скриншоте видно, что модули загружаются, регистрируют свои сервисы, инициализируются, после чего выполняются действия журналирования, валидации, демонстрации DI, времён жизни, экспорта и формирования сводки. В завершение проверяется наличие файла **grades.txt**.

```
npm start
```

```
● zhilovilya@zhilovilya-mac task1 % cd courses_pr2
● zhilovilya@zhilovilya-mac Courses_pr2 % npm start

> gradejournal@1.0.0 start
> node src/main.js

=====
[INFO] Запуск модульного приложения «Электронный журнал оценок»
=====

[INFO] Загрузка модулей из конфигурации...
Загрузка модуля: core.js
Загрузка модуля: logging.js
Загрузка модуля: validation.js
```

Проверка испытаниями

Реализованы три теста для проверки работы загрузчика модулей: корректный порядок запуска при разных наборах зависимостей, понятная ошибка при отсутствии нужного модуля и обнаружение циклической зависимости. Тесты запускаются стандартным средством Node.js без сторонних библиотек.

```
npm test
```

```

zhilovilya@zhilovilya-mac Courses_pr2 % npm test

> gradejournal@1.0.0 test
> node --test

✓ Порядок запуска учитывает зависимости (1.618791ms)
✓ Отсутствующий модуль даёт понятную ошибку (0.236875ms)
✓ Цикл зависимостей обнаруживается (0.095458ms)
i tests 3
i suites 0
i pass 3
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 87.143875
zhilovilya@zhilovilya-mac Courses_pr2 %

```

Сценарий отсутствующего модуля

Чтобы убедиться, что приложение корректно реагирует на отсутствие необходимого модуля, в `config/modules.json` достаточно оставить только зависимый модуль и убрать тот, от которого он зависит. Например, оставить `report.js` без `core.js` и `export.js`. Приложение завершит работу с понятным сообщением о том, какого именно модуля не хватает.

```

zhilovilya@zhilovilya-mac Courses_pr2 % npm start
[INFO] Загрузка модулей из конфигурации...
Загрузка модуля: logging.js
Загрузка модуля: validation.js
Загрузка модуля: report.js
Загрузка модуля: di-demo.js
Загрузка модуля: lifetime-demo.js
[SUCCESS] Загружено модулей: 5
[Logging] Загружен (зависит от: Core)
[Validation] Загружен (зависит от: Core)
[Report] Загружен (зависит от: Core, Export)
[DIIDemo] Загружен (зависит от: Core)
[LifetimeDemo] Загружен (зависит от: Core)

[INFO] Проверка зависимостей модулей...
=====
[ERROR] Ошибка: Не хватает модуля для зависимости, модуль Logging требует Core
ModuleLoadError: Не хватает модуля для зависимости, модуль Logging требует Core
    at buildOrder (file:///Users/zhilovilya/University/Framework/Task1/Courses_pr2/src/moduleLoader.js:57:15)
    at main (file:///Users/zhilovilya/University/Framework/Task1/Courses_pr2/src/main.js:50:21)
=====
zhilovilya@zhilovilya-mac Courses_pr2 %

```

Сценарий циклических зависимостей

Для проверки сценария циклических зависимостей в репозитории есть пара модулей `circular-a.js` и `circular-b.js`, которые ссылаются друг на друга. Если добавить их обоих в `config/modules.json`, приложение остановится и сообщит, какие модули образовали цикл.

```
zhilovilya@zhilovilya-mac Courses_pr2 % npm start
Загрузка модуля: circular-a.js
Загрузка модуля: circular-b.js
[SUCCESS] Загружено модулей: 9
[Core] Загружен
[Logging] Загружен (зависит от: Core)
[Validation] Загружен (зависит от: Core)
[Export] Загружен (зависит от: Core, Validation)
[Report] Загружен (зависит от: Core, Export)
[DIDemo] Загружен (зависит от: Core)
[LifetimeDemo] Загружен (зависит от: Core)
[CircularA] Загружен (зависит от: CircularB)
[CircularB] Загружен (зависит от: CircularA)

[INFO] Проверка зависимостей модулей...
=====
[ERROR] Ошибка: Обнаружена циклическая зависимость модулей, проблемные модули CircularA, CircularB
ModuleLoadError: Обнаружена циклическая зависимость модулей, проблемные модули CircularA, CircularB
    at buildOrder (file:///Users/zhilovilya/University/Framework/Task1/Courses_pr2/src/moduleLoader.js:99:11)
    at main (file:///Users/zhilovilya/University/Framework/Task1/Courses_pr2/src/main.js:50:21)
=====
zhilovilya@zhilovilya-mac Courses_pr2 %
```

Расширение приложения

При добавлении нового модуля достаточно:

1. Создать файл в папке **modules**.
2. Реализовать контракт (поля **name**, **requires**, методы **register**, **init**).
3. Добавить имя файла в **config/modules.json**.

Ядро автоматически:

- Загрузит модуль через динамический импорт.
- Проверит его зависимости.
- Включит его в порядок запуска.
- Предоставит контейнер DI для регистрации сервисов.
- Выполнит его действие.

Ни одна строка кода в папке **src** не требует изменений.

Запуск

```
npm install
npm start
npm test
```